TP5 : Programmation Python/SQL

Introduction :

Cette partie du TP porte sur la mise en place d'une interface permettant d'interagir avec une base de données relationnelle dédiée à la gestion d'une bibliothèque. L'objectif est d'automatiser différentes opérations de manipulation des données, telles que la consultation, l'ajout ou la suppression d'enregistrements.

L'utilisation d'une bibliothèque d'accès aux bases de données permet d'établir une connexion avec le serveur PostgreSQL et d'exécuter dynamiquement des requêtes SQL. Cette approche facilite l'intégration des opérations sur la base de données au sein d'un programme structuré en plusieurs fonctions réutilisables.

Dans un premier temps, des fonctions génériques sont définies afin de gérer la connexion, l'exécution des requêtes et l'affichage des résultats. Ces éléments constituent une base fonctionnelle permettant la manipulation des tables de manière fiable.

Dans un second temps, des fonctions spécifiques sont développées afin de répondre à des besoins particuliers liés à la gestion de la bibliothèque, comme la consultation des emprunts d'un abonné ou l'ajout d'un livre associé à un auteur. L'ensemble est finalement testé à l'aide d'un programme principal proposant un menu interactif facilitant l'accès aux différentes fonctionnalités.

Exercice 5.1 : Gestion de la bibliothèque

Question 1 : Voici la liste des fonctions que vous devez rédiger :

- ouvrirConnexion () retourne la connection et le curseur associés à la base de données.

```
def ouvrirConnexion():
    try:
        conn = psycopg2.connect(
            host="aquabdd",
            dbname="etudiants",
            user="12501457",
            password="100465169FK"
            )
        cur = conn.cursor()
        print("la connection est reussi")

        return conn, cur
    except:
        print("la connextion a echoué")


ouvrirConnexion()
```

- executerRequete (requete) exécute la requête requete.

```
def executerRequete(requete):
    cur.execute(requete)
```

- afficherResultat () affiche le contenu du résultat d'une requête déjà exécutée.

```
def afficherResultat():
    lignes = cur.fetchall()
    for ligne in lignes:
        print(ligne)
```

- afficherRequete (requete) exécute une requete et affiche son résultat.

```
def afficherRequete(requete):
    executerRequete(requete)
    afficherResultat()
```

- afficherTable (table) affiche le contenu de la table.

```
def afficherTable(table):
    afficherRequete(f"SELECT * FROM {table};")
```

- supprimerTuple (table ,nomId , valeurId) supprime dans la table un tuple dont l'identifiant a pour nom nomId et pour valeur valeurId.

```
def supprimerTuple(conn, cur, table, nomId, valeurId):

    try:
        if table == "livre" and nomId == "id_livre":

            # 1. emprunts
            cur.execute("""
                DELETE FROM emprunte
                WHERE id_exemplaire IN (
                    SELECT id_exemplaire FROM exemplaire WHERE id_livre = %s
                );
            """, (valeurId,))

            # 2. exemplaires
            cur.execute(
                "DELETE FROM exemplaire WHERE id_livre = %s;",
                (valeurId,)
            )

            # 3. relations auteur
            cur.execute(
                "DELETE FROM ecrit WHERE id_livre = %s;",
                (valeurId,)
            )

            # 4. decrit
            cur.execute(
                "DELETE FROM decrit WHERE id_livre = %s;",
                (valeurId,)
            )

            # 5. correspond a (NOUVEAU)
            cur.execute(
                "DELETE FROM correspond WHERE id_livre = %s;",
                (valeurId,)
            )

            # 6. livre
            cur.execute(
                "DELETE FROM livre WHERE id_livre = %s;",
                (valeurId,)
            )
```

```
        else:
            cur.execute(
                f"DELETE FROM {table} WHERE {nomId} = %s;",
                (valeurId,)
            )

        conn.commit()
        print("Suppression effectuée")

    except Exception as e:
        conn.rollback()
        print("Erreur lors de la suppression :", e)
```

- ajouterTuple (table , attributs) ajoute un tuple à la table avec les attributs spécifiés.

```
def ajouterTuple(conn, cur, table, attributs):
    """Ajoute un tuple à la table avec les attributs spécifiés.
    'attributs' est un dictionnaire {colonne: valeur}."""
    colonnes     = ", ".join(attributs.keys())
    placeholders = ", ".join(["%s"] * len(attributs))
    valeurs      = tuple(attributs.values())
    requete      = f"INSERT INTO {table} ({colonnes}) VALUES ({placeholders});"

    try:
        cur.execute(requete, valeurs)
        conn.commit()
        print(f"Tuple ajouté dans la table {table}.")
    except psycopg2.IntegrityError as e:
        conn.rollback()
        print("Violation de contrainte (clé primaire ou autre) :", e)
    except psycopg2.Error as e:
        conn.rollback()
        print("Erreur lors de l'ajout :", e)
```

- terminerConnection () met fin à la connection. Avant de fermer la connection, utiliser sa méthode commit () pour valider les modifications.

```
def terminerConnexion():
    conn.commit()
    cur.close()
    conn.close()
```

Test :

- ouvrirConnexion () :

```
12501457@r20210:/iutv/Mes_Montages/12501457$ ./tp5.py
Connexion réussie

=== MENU ===
1. Afficher une table
2. Requête SQL
3. Supprimer un tuple
4. Ajouter un tuple
5. Emprunts d'un abonné
6. Ajouter un livre et son auteur
7. Quitter
Choix :
```

- executerRequete(requete) et afficherResultat() et afficherRequete(requete) :

```
=== MENU ===
1. Afficher une table
2. Requête SQL
3. Supprimer un tuple
4. Ajouter un tuple
5. Emprunts d'un abonné
6. Ajouter un livre et son auteur
7. Quitter
Choix : 2
SQL : SELECT * FROM livre
(1, 'IdÃ©es noires', 1931)
(3, 'Lear / La mer', 1953)
(4, 'Le pion blanc des prÃ©sages', 1895)
(5, "MÃ©tro pour l'enfer", 1964)
(6, 'La reine des sortilÃ¨ges', 1845)
(7, 'Alexandre Nevsky', 1820)
(8, "L'homme qui rit", 2001)
(9, 'Les travailleurs de la mer', 1842)
(10, 'Hernani', 1830)
(11, 'Le rÃ©seau', 1974)
(12, 'Le rÃ©seau', 1982)
(13, 'Test', 1967)
```

- afficherTable(table) :

```
=== MENU ===
1. Afficher une table
2. Requête SQL
3. Supprimer un tuple
4. Ajouter un tuple
5. Emprunts d'un abonné
6. Ajouter un livre et son auteur
7. Quitter
Choix : 1
Table : motcle
(1, 'mathematiques')
(2, 'fiction')
(3, 'theorie')
(4, 'univers')
(6, 'cinema')
(67, 'test')
```

- supprimerTuple (table, nomId , valeurId) :

```
=== MENU ===
1. Afficher une table
2. Requête SQL
3. Supprimer un tuple
4. Ajouter un tuple
5. Emprunts d'un abonné
6. Ajouter un livre et son auteur
7. Quitter
Choix : 1
Table : motcle
(1, 'mathematiques')
(2, 'fiction')
(3, 'theorie')
(4, 'univers')
(6, 'cinema')
(67, 'test')

=== MENU ===
1. Afficher une table
2. Requête SQL
3. Supprimer un tuple
4. Ajouter un tuple
5. Emprunts d'un abonné
6. Ajouter un livre et son auteur
7. Quitter
Choix : 3
Table : motcle
Nom ID : id_motcle
Valeur : 67
Suppression effectuée
```

```
=== MENU ===
1. Afficher une table
2. Requête SQL
3. Supprimer un tuple
4. Ajouter un tuple
5. Emprunts d'un abonné
6. Ajouter un livre et son auteur
7. Quitter
Choix : 1
Table : motcle
(1, 'mathematiques')
(2, 'fiction')
(3, 'theorie')
(4, 'univers')
(6, 'cinema')
```

- ajouterTuple (table , attributs) ajoute un tuple à la table avec les attributs spécifiés

```
=== MENU ===
1. Afficher une table
2. Requête SQL
3. Supprimer un tuple
4. Ajouter un tuple
5. Emprunts d'un abonné
6. Ajouter un livre et son auteur
7. Quitter
Choix : 4
Dans quelle table ajouter ? motcle
Combien de colonnes à remplir ? 2
Nom de la colonne : id_motcle
Valeur : 67
Nom de la colonne : motcle
Valeur : test
Tuple ajouté dans la table motcle.

=== MENU ===
1. Afficher une table
2. Requête SQL
3. Supprimer un tuple
4. Ajouter un tuple
5. Emprunts d'un abonné
6. Ajouter un livre et son auteur
7. Quitter
Choix : 1
Table : motcle
(1, 'mathematiques')
(2, 'fiction')
(3, 'theorie')
(4, 'univers')
(6, 'cinema')
(67, 'test')
```

Question 2 : Écrire une fonction empruntsAbonne affichant la liste des titres de livres empruntés par un abonné de la bibliothèque d'identifiant id_abonne.

```
=== MENU ===
1. Afficher une table
2. Requête SQL
3. Supprimer un tuple
4. Ajouter un tuple
5. Emprunts d'un abonné
6. Ajouter un livre et son auteur
7. Quitter
Choix : 5
ID abonné : 1
Livres empruntés par l'abonné 1 :
  - Alexandre Nevsky
  - IdÃ©es noires
  - Les travailleurs de la mer
```

```python
def empruntsAbonne(cur, id_abonne):
    """Affiche la liste des titres de livres empruntés par l'abonné id_abonne."""
    requete = """
        SELECT DISTINCT l.titre
        FROM livre l
        JOIN exemplaire e  ON l.id_livre      = e.id_livre
        JOIN emprunte   em ON e.id_exemplaire = em.id_exemplaire
        WHERE em.id_abonne = %s;
    """
    try:
        cur.execute(requete, (id_abonne,))
        lignes = cur.fetchall()

        if lignes:
            print(f"Livres empruntés par l'abonné {id_abonne} :")
            for ligne in lignes:
                print(f"  - {ligne[0]}")
        else:
            print(f"Aucun emprunt pour l'abonné {id_abonne}.")

    except psycopg2.Error as e:
        print("Erreur lors de la récupération des emprunts :", e)
```

La fonction empruntsAbonne permet de récupérer et d'afficher les titres des livres actuellement empruntés par un abonné spécifique à partir de son identifiant. Pour extraire cette information, le code exécute une requête SQL utilisant des jointures (JOIN) entre les tables livre, exemplaire et emprunte, ce qui est nécessaire car le titre du livre et l'identifiant de l'abonné ne se trouvent pas dans la même table. L'utilisation du mot-clé DISTINCT dans la requête garantit que chaque titre n'apparaît qu'une seule fois, même si l'abonné possède plusieurs exemplaires du même ouvrage. Une fois les données récupérées avec fetchall(), le programme vérifie si la liste est vide ou non pour soit lister les titres un par un, soit informer qu'aucun emprunt n'est en cours pour cet utilisateur.

Question 3 : Écrire une fonction ajouterLivreAuteur (titre ,date ,nom , prenom) permettant d'ajouter un livre et son auteur à la base. Il faudra vérifier que le même livre du même auteur n'existe pas déjà.

Si ce n'est pas le cas, il faudra rajouter le livre dans la table livre.

Si l'auteur n'existe pas déjà il devra être créé. Enfin, il faudra rajouter le tuple (id_auteur , id_livre) dans la table ecrit.

```
=== MENU ===
1. Afficher une table
2. Requête SQL
3. Supprimer un tuple
4. Ajouter un tuple
5. Emprunts d'un abonné
6. Ajouter un livre et son auteur
7. Quitter
Choix : 6
Titre : Test
Date de création : 1967
Nom de l'auteur : TEST
Prénom de l'auteur : Test
Livre « Test » et auteur Test TEST ajoutés.

=== MENU ===
1. Afficher une table
2. Requête SQL
3. Supprimer un tuple
4. Ajouter un tuple
5. Emprunts d'un abonné
6. Ajouter un livre et son auteur
7. Quitter
Choix : 1
Table : livre
(1, 'IdÃ©es noires', 1931)
(3, 'Lear / La mer', 1953)
(4, 'Le pion blanc des prÃ©sages', 1895)
(5, "MÃ©tro pour l'enfer", 1964)
(6, 'La reine des sortilÃ¨ges', 1845)
(7, 'Alexandre Nevsky', 1820)
(8, "L'homme qui rit", 2001)
(9, 'Les travailleurs de la mer', 1842)
(10, 'Hernani', 1830)
(11, 'Le rÃ©seau', 1974)
(12, 'Le rÃ©seau', 1982)
(13, 'Test', 1967)

=== MENU ===
1. Afficher une table
2. Requête SQL
3. Supprimer un tuple
4. Ajouter un tuple
5. Emprunts d'un abonné
6. Ajouter un livre et son auteur
7. Quitter
Choix :
```

```python
def ajouterLivreAuteur(conn, cur, titre, date, nom, prenom):
    """Ajoute un livre et son auteur à la base.
    - Vérifie qu'un même livre du même auteur n'existe pas déjà.
    - Crée l'auteur s'il n'existe pas.
    - Insère le livre et le lien (id_auteur, id_livre) dans ecrit."""
    try:
        # 1. Recherche de l'auteur
        cur.execute(
            "SELECT id_auteur FROM auteur WHERE nom = %s AND prenom = %s;",
            (nom, prenom)
        )
        res = cur.fetchone()

        if res:
            id_auteur = res[0]
        else:
            cur.execute("SELECT MAX(id_auteur) FROM auteur;")
            max_id    = cur.fetchone()[0]
            id_auteur = (max_id + 1) if max_id else 1
            cur.execute(
                "INSERT INTO auteur VALUES (%s, %s, %s);",
                (id_auteur, nom, prenom)
            )

        # 2. Vérification du doublon livre/auteur
        cur.execute("""
            SELECT l.id_livre
            FROM livre l
            JOIN ecrit e ON l.id_livre = e.id_livre
            WHERE l.titre = %s AND e.id_auteur = %s;
        """, (titre, id_auteur))

        if cur.fetchone():
            print("Ce livre existe déjà pour cet auteur.")
            return

        # 3. Insertion du livre
        cur.execute("SELECT MAX(id_livre) FROM livre;")
        max_id   = cur.fetchone()[0]
        id_livre = (max_id + 1) if max_id else 1

        cur.execute(
            "INSERT INTO livre VALUES (%s, %s, %s);",
            (id_livre, titre, date)
        )
```

```
        # 3. Insertion du livre
        cur.execute("SELECT MAX(id_livre) FROM livre;")
        max_id   = cur.fetchone()[0]
        id_livre = (max_id + 1) if max_id else 1

        cur.execute(
            "INSERT INTO livre VALUES (%s, %s, %s);",
            (id_livre, titre, date)
        )

        # 4. Liaison auteur ↔ livre
        cur.execute(
            "INSERT INTO ecrit VALUES (%s, %s);",
            (id_auteur, id_livre)
        )

        conn.commit()
        print(f"Livre « {titre} » et auteur {prenom} {nom} ajoutés.")

    except psycopg2.IntegrityError as e:
        conn.rollback()
        print("Violation de contrainte lors de l'ajout du livre :", e)
    except psycopg2.Error as e:
        conn.rollback()
        print("Erreur lors de l'ajout du livre :", e)
```

La fonction ajouterLivreAuteur sert à automatiser l'entrée des données tout en respectant les contraintes d'intégrité de notre base. En gros, pour coller au diagramme de classes, le script commence par checker si l'auteur existe déjà avec son nom et son prénom pour pas faire de doublons inutiles dans la table Auteur. Si le mec n'est pas dans la base, on lui crée un identifiant unique en faisant un petit MAX(idAuteur) + 1.

Ensuite, on fait la même logique pour la table Livre : on crée le bouquin avec son titre et sa date de création. La dernière étape, c'est de lier les deux : on récupère les clés primaires (idAuteur et idLivre) et on les injecte dans la table de relation écrit. C'est ce qui permet de gérer proprement la relation entre l'œuvre et l'écrivain, exactement comme on l'a vu sur notre schéma relationnel. En résumé, ça permet de gérer des relations complexes sans se prendre la tête tout en gardant une base propre et cohérente.

Conclusion :

En conclusion, cette cinquième partie nous a permis de mettre en pratique les notions fondamentales des bases de données relationnelles à travers l'exploitation concrète de la base de données de la bibliothèque. Nous avons utilisé le langage SQL afin d'interroger les données, en appliquant différentes opérations vues en cours comme les sélections, projections, jointures ainsi que les regroupements.

Ce travail nous a également permis de mieux comprendre l'organisation des données au sein d'un SGBD, notamment l'importance des clés primaires et étrangères pour garantir la cohérence et l'intégrité des informations. L'utilisation de requêtes a montré comment extraire efficacement des informations pertinentes à partir de plusieurs tables liées entre elles.

Pour réaliser cette partie, nous avons utilisé plusieurs éléments essentiels :

- le SGBD PostgreSQL pour gérer la base de données,
- l'outil psql pour exécuter les commandes SQL,
- le langage SQL (SELECT, WHERE, JOIN, GROUP BY, etc.),
- les tables relationnelles issues du schéma (livre, auteur, exemplaire, abonné, etc.),
- les fichiers de données (.txt) au format CSV avec séparateur « ; » pour remplir la base,
- ainsi que les notions théoriques du cours comme les jointures, les relations et les contraintes d'intégrité.

Ainsi, cette partie nous a permis de consolider nos compétences en manipulation de bases de données et de mieux comprendre le lien entre la théorie et la pratique.